

Punto 2.12 - Clean Architecture y Arquitectura Hexagonal

Audiencia: Desarrolladores que ya han completado el modelado acumulativo 2.1-2.11.

Proyecto: 2.12 parte de 2.11 y refactoriza el mismo sistema en Domain, Application, Infrastructure y Console. El refactor cambia la organización del código, no el dominio ni el objetivo de persistencia.

Objetivos de aprendizaje

- Separar dominio y persistencia.
- Mantener EF Core fuera de Domain y Application.
- Definir puertos de repositorio y unidad de trabajo.
- Implementar adaptadores en Infrastructure.
- Registrar dependencias desde una extensión de IServiceCollection.
- Conservar el historial de migraciones y el comportamiento del modelo.

Paso 1: Abrir la solución multiproyecto

```
cd M02/PROYECTO/2.12
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

La solución contiene cuatro proyectos:

```
src/
├─ AceriaData.Domain
├─ AceriaData.Application
├─ AceriaData.Infrastructure
└─ AceriaData.Console
```

Paso 2: Mantener el dominio independiente de EF Core

```
namespace AceriaData.Domain.Entities;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
```

```

    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
}

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class CertificadoCalidad
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    public string OrganismoCertificador { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    public decimal CantidadUtilizada { get; set; }
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

```

Las entidades están en Domain y no necesitan `DbContext`, `DbSet`, `IEntityTypeConfiguration` ni referencias a paquetes de EF Core.

Paso 3: Definir los puertos de aplicación

```
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    List<OrdenFabricacion> ObtenerTodas();
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}

public interface IUnidadDeTrabajo : IDisposable
{
    IOrdenRepositorio Ordenes { get; }
    int Guardar();
}
```

Línea 1: `IOrdenRepositorio` → expresa operaciones que necesita la aplicación sin exponer EF Core.

Línea 2: `IUnidadDeTrabajo` → coordina repositorios y el guardado como abstracción de aplicación.

Paso 4: Crear un caso de uso

```
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.UseCases;

public sealed class CrearOrdenUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public CrearOrdenUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public int Ejecutar(string numeroOrden, string cliente)
    {
        _unidad Ordenes.Agregar(new OrdenFabricacion
        {
            NumeroOrden = numeroOrden,
            Cliente = cliente,
            FechaCreacion = DateTime.UtcNow,
            Estado = "Pendiente"
        });
        return _unidad.Guardar();
    }
}
```

El caso de uso depende de interfaces de Application y de entidades de Domain. No referencia `Microsoft.EntityFrameworkCore`.

Paso 5: Colocar EF Core y toda la configuración acumulada en Infrastructure

El `AceriaDbContext` real de 2.12 contiene los `DbSet` y descubre las configuraciones de Infrastructure mediante `ApplyConfigurationsFromAssembly`:

```

using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Persistence;

public sealed class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(AceriaDbContext).Assembly);
    }
}

```

Línea 1: `AceriaDbContext : DbContext` → mantiene EF Core exclusivamente en Infrastructure.

Línea 2: `DbSet<...>` → expone las siete entidades acumuladas del modelo 2.1-2.11.

Línea 3: `OnModelCreating` → centraliza la construcción del modelo relacional.

Línea 4: `ApplyConfigurationsFromAssembly(typeof(AceriaDbContext).Assembly)` → descubre automáticamente todas las clases `IEntityTypeConfiguration<T>` del ensamblado.

Línea 5: `base.OnModelCreating(modelBuilder)` → conserva el comportamiento base antes de aplicar la configuración del dominio.

Paso 5.1: Conservar las configuraciones Fluent API acumuladas

Mover el DbContext no basta: para que 2.12 represente realmente el mismo modelo que 2.11 hay que conservar relaciones, claves alternativas, índices, restricciones, valores por defecto y filtros globales. Estos son los archivos reales del estado 2.12.

OrdenFabricacionConfiguration.cs

```

using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace AceriaData.Infrastructure.Persistence.Configurations;

public sealed class OrdenFabricacionConfiguration : IEntityTypeConfiguration<OrdenFabricacion>
{
    public void Configure(EntityTypeBuilder<OrdenFabricacion> b)
    {
        b.ToTable("OrdenesFabricacion");
        b.HasKey(x => x.Id);
        b.HasAlternateKey(x => x.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
        b.Property(x => x.NumeroOrden).IsRequired().HasMaxLength(50);
        b.Property(x => x.Cliente).IsRequired().HasMaxLength(200);
        b.Property(x => x.FechaCreacion).HasDefaultValueSql("GETDATE()");
        b.Property(x => x.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
        b.Property(x => x.Observaciones).HasMaxLength(500);
        b.HasQueryFilter(x => !x.IsDeleted && x.Estado != "Cancelada");
        b.HasIndex(x => x.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");
        b.HasIndex(x => new { x.Cliente,
x.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");
        b.HasIndex(x => x.FechaEntrega).HasFilter("[Estado] =
'Pendiente'").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");
        b.HasIndex(x => x.Estado).IncludeProperties(x => new { x.NumeroOrden, x.Cliente,
x.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");
    }
}

```

PlanchaAceroConfiguration.cs

```

using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace AceriaData.Infrastructure.Persistence.Configurations;

public sealed class PlanchaAceroConfiguration : IEntityTypeConfiguration<PlanchaAcero>
{
    public void Configure(EntityTypeBuilder<PlanchaAcero> b)
    {
        b.ToTable("PlanchasAcero", t =>
        {
            t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");
        });
        b.HasKey(x => x.Id);
        b.Property(x => x.Peso).HasPrecision(18, 3);
        b.Property(x => x.Activa).HasDefaultValue(true);
        b.HasOne(x => x.Orden).WithMany(x => x.Planchas).HasForeignKey(x =>
x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();
        b.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");
        b.HasIndex(x => x.Espesor).HasFilter("[Activa] =
1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");
        b.HasQueryFilter(x => !x.IsDeleted && x.Activa);
    }
}

```

ModeloConfiguration.cs

```

using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace AceriaData.Infrastructure.Persistence.Configurations;

public sealed class AleacionConfiguration : IEntityTypeConfiguration<Aleacion>
{

```

```

    public void Configure(EntityTypeBuilder<Aleacion> b)
    {
        b.ToTable("Aleaciones", t =>
        {
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5");
        });
        b.HasKey(x => x.Id);
        b.HasAlternateKey(x => x.Codigo).HasName("AK_Aleaciones_Codigo");
        b.Property(x => x.Nombre).IsRequired().HasMaxLength(100);
        b.Property(x => x.Codigo).IsRequired().HasMaxLength(20);
        b.Property(x => x.Descripcion).HasMaxLength(500);
        b.HasIndex(x => x.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");
        b.HasIndex(x => new { x.PorcentajeCarbono, x.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");
        b.HasQueryFilter(x => !x.IsDeleted);
    }
}

public sealed class EstadoOrdenConfiguration : IEntityTypeConfiguration<EstadoOrden>
{
    public void Configure(EntityTypeBuilder<EstadoOrden> b)
    {
        b.ToTable("EstadosOrden");
        b.HasKey(x => x.Id);
        b.Property(x => x.Nombre).IsRequired().HasMaxLength(50);
        b.Property(x => x.Descripcion).HasMaxLength(250);
        b.Property(x => x.Activo).HasDefaultValue(true);
        b.HasQueryFilter(x => !x.IsDeleted && x.Activo);
    }
}

public sealed class DetalleOrdenConfiguration : IEntityTypeConfiguration<DetalleOrden>
{
    public void Configure(EntityTypeBuilder<DetalleOrden> b)
    {
        b.ToTable("DetallesOrden");
        b.HasKey(x => x.Id);
        b.Property(x => x.ComposicionQuimica).IsRequired().HasMaxLength(200);
        b.Property(x => x.Notas).HasMaxLength(500);
        b.HasOne(x => x.Orden).WithOne(x => x.Detalle).HasForeignKey<DetalleOrden>(x => x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();
        b.HasQueryFilter(x => !x.Orden.IsDeleted && x.Orden.Estado != "Cancelada");
    }
}

public sealed class CertificadoCalidadConfiguration : IEntityTypeConfiguration<CertificadoCalidad>
{
    public void Configure(EntityTypeBuilder<CertificadoCalidad> b)
    {
        b.ToTable("CertificadosCalidad");
        b.HasKey(x => x.Id);
        b.HasAlternateKey(x => x.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");
        b.Property(x => x.NumeroCertificado).IsRequired().HasMaxLength(50);
        b.Property(x => x.OrganismoCertificador).IsRequired().HasMaxLength(100);
        b.HasIndex(x => x.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");
        b.HasOne(x => x.Orden).WithOne(x => x.Certificado).HasForeignKey<CertificadoCalidad>(x => x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();
        b.HasQueryFilter(x => !x.Orden.IsDeleted && x.Orden.Estado != "Cancelada");
    }
}

public sealed class OrdenAleacionConfiguration : IEntityTypeConfiguration<OrdenAleacion>
{
    public void Configure(EntityTypeBuilder<OrdenAleacion> b)
    {
        b.ToTable("OrdenesAleaciones");
        b.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        b.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
        b.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        b.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
    }
}

```

```

        b.HasOne(x => x.Orden).WithMany(x => x.OrdenesAleaciones).HasForeignKey(x =>
x.OrdenFabricacionId).OnDelete(DeleteBehavior.Cascade);
        b.HasOne(x => x.Aleacion).WithMany(x => x.OrdenesAleaciones).HasForeignKey(x =>
x.AleacionId).OnDelete(DeleteBehavior.Restrict);
        b.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");
        b.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] =
'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");
        b.HasQueryFilter(x => x.EstadoRelacion == "Activa" && !x.Orden.IsDeleted && !
x.Aleacion.IsDeleted);
    }
}

```

Línea 1: `HasAlternateKey` → conserva los identificadores naturales configurados en 2.8.

Línea 2: `HasIndex` / `HasFilter` / `IncludeProperties` → conserva los índices simples, compuestos, filtrados y con columnas incluidas introducidos en 2.9.

Línea 3: `HasCheckConstraint` → mantiene las reglas de integridad que deben cumplirse también fuera de la aplicación.

Línea 4: `HasQueryFilter` → conserva los filtros globales y el Soft Delete de 2.10-2.11.

Línea 5: `HasOne` / `WithMany` / `WithOne` → conserva las relaciones uno-a-muchos, uno-a-uno y muchos-a-muchos acumuladas.

| Paso 6: Implementar y registrar los adaptadores de Infrastructure

Antes de registrar los puertos hay que implementar sus adaptadores. El archivo real `Repositories.cs` contiene tanto `OrdenRepositorio` como `UnidadDeTrabajo`:

```

using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(x => x.NumeroOrden == numeroOrden);
    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(x =>
x.Id).ToList();
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
}

public sealed class UnidadDeTrabajo : IUnidadDeTrabajo
{
    private readonly AceriaDbContext _context;
    private IOrdenRepositorio? _ordenes;
    public UnidadDeTrabajo(AceriaDbContext context) => _context = context;
    public IOrdenRepositorio Ordenes => _ordenes ??= new OrdenRepositorio(_context);
    public int Guardar() => _context.SaveChanges();
    public void Dispose() => _context.Dispose();
}

```

Línea 1: `OrdenRepositorio : IOrdenRepositorio` → implementa en Infrastructure el puerto definido por Application.

Línea 2: `ObtenerPorNumero` → traduce una operación de aplicación a una consulta de EF Core.

Línea 3: `Agregar` / `Eliminar` → modifican el estado del DbContext sin llamar por sí mismos a `SaveChanges`.

Línea 4: `UnidadDeTrabajo : IUnidadDeTrabajo` → coordina el repositorio y el `DbContext` compartido.

Línea 5: `Guardar() => _context.SaveChanges()` → concentra la confirmación de cambios en la unidad de trabajo.

Después se registran SQL Server y los adaptadores con la extensión real `AddAceriaInfrastructure`:

```
using AceriaData.Application.Interfaces;
using AceriaData.Infrastructure.Persistence;
using AceriaData.Infrastructure.Repositories;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;

namespace AceriaData.Infrastructure;

public static class DependencyInjection
{
    public static IServiceCollection AddAceriaInfrastructure(this IServiceCollection services, string
connectionString)
    {
        services.AddDbContext<AceriaDbContext>(o => o.UseSqlServer(connectionString));
        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();
        return services;
    }
}
```

Línea 1: `AddDbContext<AceriaDbContext>` → registra el contexto con el proveedor de SQL Server.

Línea 2: `AddScoped<IOrdenRepositorio, OrdenRepositorio>()` → enlaza el puerto de Application con su adaptador de Infrastructure.

Línea 3: `AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>()` → registra la unidad de trabajo con el mismo ciclo de vida scoped.

Paso 7: Ejecutar desde la capa de entrada

```
using AceriaData.Application.UseCases;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var cs = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(cs);
services.AddScoped<CrearOrdenUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true,
ValidateScopes = true });
using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var crear = scope.ServiceProvider.GetRequiredService<CrearOrdenUseCase>();
crear.Ejecutar("OF-M2-HEX-0001", "Cliente Arquitectura");
var orden = context.OrdenesFabricacion.Single();
Console.WriteLine($"2.12 OK | {orden.NumeroOrden} | {orden.Cliente}");
```

La consola construye el contenedor, resuelve el caso de uso y ejecuta la aplicación. El resultado esperado contiene `2.12 OK`.

▮ Paso 8: Validar migraciones desde la nueva ubicación

Para que las herramientas de EF Core puedan crear el contexto de forma reproducible en tiempo de diseño, Infrastructure contiene la factory real:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Design;

namespace AceriaData.Infrastructure.Persistence;

public sealed class AceriaDesignTimeDbContextFactory : IDesignTimeDbContextFactory<AceriaDbContext>
{
    public AceriaDbContext CreateDbContext(string[] args)
    {
        var options = new DbContextOptionsBuilder<AceriaDbContext>()
            .UseSqlServer("Server=(localdb)\
\MSSQLLocalDB;Database=AceriaDB;Trusted_Connection=True;TrustServerCertificate=True;")
            .Options;
        return new AceriaDbContext(options);
    }
}
```

Línea 1: `IDesignTimeDbContextFactory<AceriaDbContext>` → ofrece a `dotnet ef` una forma explícita de construir el contexto.

Línea 2: `UseSqlServer(...)` → usa la misma instancia LocalDB y la misma base AceriaDB del curso.

Línea 3: `return new AceriaDbContext(options)` → devuelve el contexto configurado sin depender del flujo interactivo de Console.

Con la factory y las migraciones trasladadas a Infrastructure, se valida el historial y se ejecuta el estado final:

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/
AceriaData.Console
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

El historial de migraciones de 2.1-2.11 se conserva y se añade el estado de arquitectura 2.12. El segundo comando debe terminar mostrando el marcador `2.12 OK`.

▮ Explicación línea a línea del estado 2.12 por capas

A diferencia de 2.1-2.11, el estado 2.12 está repartido en varios proyectos y archivos. Las líneas siguientes explican **cada línea no vacía** de los once archivos fuente que la práctica muestra y que el repositorio ejecutable utiliza.

src/AceriaData.Domain/Entities.cs

Línea 1: `namespace AceriaData.Domain.Entities;` → declara el espacio de nombres `AceriaData.Domain.Entities` y sitúa el archivo en su capa arquitectónica.

Línea 3: `public class OrdenFabricacion` → declara la entidad raíz de la orden de fabricación dentro de `Domain`.

Línea 4: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 5: `public int Id { get; set; }` → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.

Línea 6: `public string NumeroOrden { get; set; } = string.Empty;` → define el identificador de negocio de la orden.

Línea 7: `public string Cliente { get; set; } = string.Empty;` → almacena el cliente asociado a la orden.

Línea 8: `public DateTime FechaCreacion { get; set; }` → almacena la fecha de creación de la orden.

Línea 9: `public DateTime? FechaEntrega { get; set; }` → almacena una fecha de entrega opcional.

Línea 10: `public string Estado { get; set; } = "Pendiente";` → almacena el estado de la orden y parte con el valor de dominio Pendiente.

Línea 11: `public string? Observaciones { get; set; }` → almacena observaciones opcionales.

Línea 12: `public bool IsDeleted { get; set; }` → mantiene la marca utilizada por Soft Delete.

Línea 13: `public DateTime? DeletedAt { get; set; }` → registra opcionalmente cuándo se produjo el borrado lógico.

Línea 14: `public List<PlanchaAcero> Planchas { get; set; } = new();` → expone la navegación de colección hacia las planchas de la orden.

Línea 15: `public DetalleOrden? Detalle { get; set; }` → expone la navegación opcional uno-a-uno hacia DetalleOrden.

Línea 16: `public CertificadoCalidad? Certificado { get; set; }` → expone la navegación opcional uno-a-uno hacia CertificadoCalidad.

Línea 17: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → expone la colección de filas de unión de la relación muchos-a-muchos.

Línea 18: `}` → cierra el bloque de código actual.

Línea 20: `public class PlanchaAcero` → declara la entidad de plancha relacionada con una orden.

Línea 21: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 22: `public int Id { get; set; }` → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.

Línea 23: `public int OrdenId { get; set; }` → almacena la clave foránea hacia OrdenFabricacion.

Línea 24: `public double Espesor { get; set; }` → almacena el espesor de la plancha.

Línea 25: `public double Ancho { get; set; }` → almacena el ancho de la plancha.

Línea 26: `public double Largo { get; set; }` → almacena el largo de la plancha.

Línea 27: `public decimal Peso { get; set; }` → almacena el peso con precisión configurada posteriormente mediante Fluent API.

Línea 28: `public bool Activa { get; set; } = true;` → indica si la plancha está activa.

Línea 29: `public bool IsDeleted { get; set; }` → mantiene la marca utilizada por Soft Delete.

Línea 30: `public DateTime? DeletedAt { get; set; }` → registra opcionalmente cuándo se produjo el borrado lógico.

Línea 31: `public OrdenFabricacion Orden { get; set; } = null!;` → expone la navegación hacia la orden principal.

Línea 32: `}` → cierra el bloque de código actual.

Línea 34: `public class Aleacion` → declara la entidad de aleación utilizada por la relación muchos-a-muchos explícita.

Línea 35: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 36: `public int Id { get; set; }` → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.

Línea 37: `public string Nombre { get; set; } = string.Empty;` → almacena el nombre funcional de la entidad.

Línea 38: `public string Codigo { get; set; } = string.Empty;` → almacena el código natural de la aleación.

Línea 39: `public double PorcentajeCarbono { get; set; }` → almacena el porcentaje de carbono validado por una restricción CHECK.

Línea 40: `public double PorcentajeManganeso { get; set; }` → almacena el porcentaje de manganeso validado por una restricción CHECK.

Línea 41: `public string? Descripcion { get; set; }` → almacena una descripción de negocio.

Línea 42: `public bool IsDeleted { get; set; }` → mantiene la marca utilizada por Soft Delete.

Línea 43: `public DateTime? DeletedAt { get; set; }` → registra opcionalmente cuándo se produjo el borrado lógico.

Línea 44: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → expone la colección de filas de unión de la relación muchos-a-muchos.

Línea 45: `}` → cierra el bloque de código actual.

Línea 47: `public class EstadoOrden` → declara la entidad que representa un estado de negocio de la orden.

Línea 48: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 49: `public int Id { get; set; }` → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.

Línea 50: `public string Nombre { get; set; } = string.Empty;` → almacena el nombre funcional de la entidad.

Línea 51: `public string Descripcion { get; set; } = string.Empty;` → almacena una descripción de negocio.

Línea 52: `public bool Activo { get; set; } = true;` → indica si el estado está activo.

Línea 53: `public bool IsDeleted { get; set; }` → mantiene la marca utilizada por Soft Delete.

Línea 54: `public DateTime? DeletedAt { get; set; }` → registra opcionalmente cuándo se produjo el borrado lógico.

Línea 55: `}` → cierra el bloque de código actual.

Línea 57: `public class DetalleOrden` → declara la entidad dependiente de la relación uno-a-uno con OrdenFabricacion.

Línea 58: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 59: `public int Id { get; set; }` → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.

Línea 60: `public int OrdenId { get; set; }` → almacena la clave foránea hacia OrdenFabricacion.

Línea 61: `public string ComposicionQuimica { get; set; } = string.Empty;` → almacena la composición química del detalle.

Línea 62: `public double TemperaturaColada { get; set; }` → almacena la temperatura de colada.

Línea 63: `public string? Notas { get; set; }` → almacena notas opcionales del detalle.

Línea 64: `public OrdenFabricacion Orden { get; set; } = null!;` → expone la navegación hacia la orden principal.

Línea 65: `}` → cierra el bloque de código actual.

Línea 67: `public class CertificadoCalidad` → declara la entidad dependiente que representa el certificado de calidad de una orden.

Línea 68: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 69: `public int Id { get; set; }` → define la clave técnica de la entidad; su configuración concreta se mantiene en Infrastructure.

Línea 70: `public int OrdenId { get; set; }` → almacena la clave foránea hacia OrdenFabricacion.

Línea 71: `public string NumeroCertificado { get; set; } = string.Empty;` → almacena el identificador natural del certificado.

Línea 72: `public DateTime FechaEmision { get; set; }` → almacena la fecha de emisión del certificado.

Línea 73: `public string OrganismoCertificador { get; set; } = string.Empty;` → almacena el organismo que emitió el certificado.

Línea 74: `public OrdenFabricacion Orden { get; set; } = null!;` → expone la navegación hacia la orden principal.

Línea 75: `}` → cierra el bloque de código actual.

Línea 77: `public class OrdenAleacion` → declara la entidad de unión explícita entre OrdenFabricacion y Aleacion.

Línea 78: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 79: `public int OrdenFabricacionId { get; set; }` → almacena la parte de la clave compuesta que referencia a OrdenFabricacion.

Línea 80: `public int AleacionId { get; set; }` → almacena la parte de la clave compuesta que referencia a Aleacion.

Línea 81: `public DateTime FechaAsignacion { get; set; } = DateTime.Now;` → almacena cuándo se creó la relación orden-aleación.

Línea 82: `public decimal CantidadUtilizada { get; set; }` → almacena la cantidad de aleación utilizada.

Línea 83: `public string EstadoRelacion { get; set; } = "Activa";` → almacena el estado lógico de la relación orden-aleación.

Línea 84: `public OrdenFabricacion Orden { get; set; } = null!;` → expone la navegación hacia la orden principal.

Línea 85: `public Aleacion Aleacion { get; set; } = null!;` → expone la navegación hacia Aleacion.

Línea 86: `}` → cierra el bloque de código actual.

src/AceriaData.Application/Interfaces.cs

Línea 1: `using AceriaData.Domain.Entities;` → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.

Línea 3: `namespace AceriaData.Application.Interfaces;` → declara el espacio de nombres AceriaData.Application.Interfaces y sitúa el archivo en su capa arquitectónica.

Línea 5: `public interface IOrdenRepositorio` → declara el puerto de persistencia de órdenes que Application necesita y que Infrastructure implementará.

Línea 6: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 7: `OrdenFabricacion? ObtenerPorId(int id);` → declara la operación para recuperar una orden por su clave técnica.

Línea 8: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara la operación para recuperar una orden por su número de negocio.

Línea 9: `List<OrdenFabricacion> ObtenerTodas();` → declara la operación que devuelve las órdenes visibles para el repositorio.

Línea 10: `void Agregar(OrdenFabricacion orden);` → declara la operación que registra una nueva orden.

Línea 11: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el caso de uso lo requiera.

Línea 12: `}` → cierra el bloque de código actual.

Línea 14: `public interface IUnidadDeTrabajo : IDisposable` → declara el puerto de unidad de trabajo que coordina el repositorio y la confirmación de cambios.

Línea 15: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 16: `IOrdenRepositorio Ordenes { get; }` → expone el repositorio de órdenes a través del puerto de unidad de trabajo.

Línea 17: `int Guardar();` → declara la operación que confirma de forma coordinada los cambios pendientes.

Línea 18: `}` → cierra el bloque de código actual.

src/AceriaData.Application/CrearOrdenUseCase.cs

Línea 1: `using AceriaData.Application.Interfaces;` → importa los puertos definidos por Application para que Infrastructure pueda implementarlos.

Línea 2: `using AceriaData.Domain.Entities;` → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.

Línea 4: `namespace AceriaData.Application.UseCases;` → declara el espacio de nombres `AceriaData.Application.UseCases` y sitúa el archivo en su capa arquitectónica.

Línea 6: `public sealed class CrearOrdenUseCase` → declara el caso de uso de Application encargado de crear una orden sin depender de EF Core.

Línea 7: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 8: `private readonly IUnidadDeTrabajo _unidad;` → guarda la dependencia del caso de uso contra la abstracción `IUnidadDeTrabajo`.

Línea 9: `public CrearOrdenUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;` → inyecta `IUnidadDeTrabajo` en el caso de uso sin acoplar Application a Infrastructure.

Línea 11: `public int Ejecutar(string numeroOrden, string cliente)` → define la operación de aplicación que crea una orden y devuelve el resultado del guardado.

Línea 12: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 13: `_unidad.Ordenes.Agregar(new OrdenFabricacion` → crea una orden a través del puerto `IOrdenRepositorio` expuesto por la unidad de trabajo.

Línea 14: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 15: `NumeroOrden = numeroOrden,` → copia al dominio el número de orden recibido por el caso de uso.

Línea 16: `Cliente = cliente,` → copia al dominio el cliente recibido por el caso de uso.

Línea 17: `FechaCreacion = DateTime.UtcNow,` → asigna una fecha de creación UTC desde la aplicación.

Línea 18: `Estado = "Pendiente"` → inicializa la nueva orden con el estado `Pendiente`.

Línea 19: `});` → cierra el inicializador o la llamada iniciada en las líneas anteriores.

Línea 20: `return _unidad.Guardar();` → confirma los cambios una única vez mediante la unidad de trabajo y devuelve las filas afectadas.

Línea 21: `}` → cierra el bloque de código actual.

Línea 22: `}` → cierra el bloque de código actual.

src/AceriaData.Infrastructure/Persistence/AceriaDbContext.cs

Línea 1: `using AceriaData.Domain.Entities;` → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.

Línea 2: `using Microsoft.EntityFrameworkCore;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 4: `namespace AceriaData.Infrastructure.Persistence;` → declara el espacio de nombres `AceriaData.Infrastructure.Persistence` y sitúa el archivo en su capa arquitectónica.

Línea 6: `public sealed class AceriaDbContext : DbContext` → declara el `DbContext` de Infrastructure y concentra el modelo EF Core.

Línea 7: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 8: `public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → recibe `DbContextOptions` por inyección y los pasa al constructor base de `DbContext`.

Línea 10: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone el `DbSet` `OrdenesFabricacion` para consultar y persistir esa entidad.

Línea 11: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone el `DbSet` `PlanchasAcero` para consultar y persistir esa entidad.

Línea 12: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone el `DbSet` `Aleaciones` para consultar y persistir esa entidad.

Línea 13: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone el `DbSet` `EstadosOrden` para consultar y persistir esa entidad.

Línea 14: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone el `DbSet` `DetallesOrden` para consultar y persistir esa entidad.

Línea 15: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone el `DbSet` `CertificadosCalidad` para consultar y persistir esa entidad.

Línea 16: `public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();` → expone el `DbSet` `OrdenesAleaciones` para consultar y persistir esa entidad.

Línea 18: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → sobrescribe `OnModelCreating` para aplicar la configuración relacional del modelo.

Línea 19: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 20: `base.OnModelCreating(modelBuilder);` → ejecuta primero la configuración base de `DbContext`.

Línea 21: `modelBuilder.ApplyConfigurationsFromAssembly(typeof(AceriaDbContext).Assembly);` → descubre y aplica automáticamente todas las implementaciones `IEntityTypeConfiguration` del ensamblado de Infrastructure.

Línea 22: `}` → cierra el bloque de código actual.

Línea 23: `}` → cierra el bloque de código actual.

src/AceriaData.Infrastructure/Persistence/Configurations/OrdenFabricacionConfiguration.cs

Línea 1: `using AceriaData.Domain.Entities;` → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.

Línea 2: `using Microsoft.EntityFrameworkCore;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 3: `using Microsoft.EntityFrameworkCore.Metadata.Builders;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 5: `namespace AceriaData.Infrastructure.Persistence.Configurations;` → declara el espacio de nombres `AceriaData.Infrastructure.Persistence.Configurations` y sitúa el archivo en su capa arquitectónica.

Línea 7: `public sealed class OrdenFabricacionConfiguration : IEntityTypeConfiguration<OrdenFabricacion>` → declara la configuración Fluent API de `OrdenFabricacion`.

Línea 8: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 9: `public void Configure(EntityTypeBuilder<OrdenFabricacion> b)` → implementa el método `Configure` de `IEntityTypeConfiguration` para describir el mapeo de la entidad.

Línea 10: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 11: `b.ToTable("OrdenesFabricacion");` → asigna la entidad a la tabla SQL indicada; cuando incluye `lambda`, configura además restricciones de tabla.

Línea 12: `b.HasKey(x => x.Id);` → define explícitamente la clave primaria de la entidad.

Línea 13: `b.HasAlternateKey(x => x.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");` → define una clave alternativa para el identificador natural y permite referenciarlo como principal.

Línea 14: `b.Property(x => x.NumeroOrden).IsRequired().HasMaxLength(50);` → marca la propiedad como requerida y limita su longitud en el esquema.

Línea 15: `b.Property(x => x.Cliente).IsRequired().HasMaxLength(200);` → marca la propiedad como requerida y limita su longitud en el esquema.

Línea 16: `b.Property(x => x.FechaCreacion).HasDefaultValueSql("GETDATE()");` → configura una propiedad cuyo valor por defecto se obtiene mediante una expresión SQL del servidor.

Línea 17: `b.Property(x => x.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → configura un valor por defecto de base de datos para la propiedad.

Línea 18: `b.Property(x => x.Observaciones).HasMaxLength(500);` → limita la longitud máxima de la columna correspondiente.

Línea 19: `b.HasQueryFilter(x => !x.IsDeleted && x.Estado != "Cancelada");` → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.

Línea 20: `b.HasIndex(x => x.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");` → crea y nombra un índice sobre la propiedad indicada.

Línea 21: `b.HasIndex(x => new { x.Cliente, x.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");` → crea un índice compuesto sobre las propiedades indicadas.

Línea 22: `b.HasIndex(x => x.FechaEntrega).HasFilter("[Estado] = 'Pendiente'").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");` → crea un índice filtrado para el subconjunto de filas indicado.

Línea 23: `b.HasIndex(x => x.Estado).IncludeProperties(x => new { x.NumeroOrden, x.Cliente, x.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");` → crea un índice con columnas incluidas para cubrir consultas frecuentes.

Línea 24: `}` → cierra el bloque de código actual.

Línea 25: `}` → cierra el bloque de código actual.

src/AceriaData.Infrastructure/Persistence/Configurations/PlanchaAceroConfiguration.cs

Línea 1: `using AceriaData.Domain.Entities;` → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.

Línea 2: `using Microsoft.EntityFrameworkCore;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 3: `using Microsoft.EntityFrameworkCore.Metadata.Builders;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 5: `namespace AceriaData.Infrastructure.Persistence.Configurations;` → declara el espacio de nombres `AceriaData.Infrastructure.Persistence.Configurations` y sitúa el archivo en su capa arquitectónica.

Línea 7: `public sealed class PlanchaAceroConfiguration : IEntityTypeConfiguration<PlanchaAcero>` → declara la configuración Fluent API de `PlanchaAcero`.

Línea 8: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 9: `public void Configure(EntityTypeBuilder<PlanchaAcero> b)` → implementa el método `Configure` de `IEntityTypeConfiguration` para describir el mapeo de la entidad.

Línea 10: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 11: `b.ToTable("PlanchasAcero", t =>` → asigna la entidad a la tabla SQL indicada; cuando incluye lambda, configura además restricciones de tabla.

Línea 12: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 13: `t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");` → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.

Línea 14: `t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");` → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.

Línea 15: `t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");` → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.

Línea 16: `t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");` → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.

Línea 17: `});` → cierra el inicializador o la llamada iniciada en las líneas anteriores.

Línea 18: `b.HasKey(x => x.Id);` → define explícitamente la clave primaria de la entidad.

Línea 19: `b.Property(x => x.Peso).HasPrecision(18, 3);` → configura precisión y escala para el valor decimal.

Línea 20: `b.Property(x => x.Activa).HasDefaultValue(true);` → configura un valor por defecto de base de datos para la propiedad.

Línea 21: `b.HasOne(x => x.Orden).WithMany(x => x.Planchas).HasForeignKey(x => x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();` → configura una relación uno-a-muchos y encadena la clave foránea y el comportamiento de borrado.

Línea 22: `b.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");` → crea un índice compuesto sobre las propiedades indicadas.

Línea 23: `b.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");` → crea un índice filtrado para el subconjunto de filas indicado.

Línea 24: `b.HasQueryFilter(x => !x.IsDeleted && x.Activa);` → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.

Línea 25: `}` → cierra el bloque de código actual.

Línea 26: `}` → cierra el bloque de código actual.

src/AceriaData.Infrastructure/Persistence/Configurations/ModeloConfiguration.cs

Línea 1: `using AceriaData.Domain.Entities;` → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.

Línea 2: `using Microsoft.EntityFrameworkCore;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 3: `using Microsoft.EntityFrameworkCore.Metadata.Builders;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 5: `namespace AceriaData.Infrastructure.Persistence.Configurations;` → declara el espacio de nombres `AceriaData.Infrastructure.Persistence.Configurations` y sitúa el archivo en su capa arquitectónica.

Línea 7: `public sealed class AleacionConfiguration : IEntityTypeConfiguration<Aleacion>` → declara la configuración Fluent API de Aleacion.

Línea 8: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 9: `public void Configure(EntityTypeBuilder<Aleacion> b)` → implementa el método `Configure` de `IEntityTypeConfiguration` para describir el mapeo de la entidad.

Línea 10: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 11: `b.ToTable("Aleaciones", t =>` → asigna la entidad a la tabla SQL indicada; cuando incluye lambda, configura además restricciones de tabla.

Línea 12: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 13: `t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");` → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.

Línea 14: `t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5");` → añade una restricción CHECK en SQL Server para reforzar una regla de integridad en la base de datos.

Línea 15: `});` → cierra el inicializador o la llamada iniciada en las líneas anteriores.

Línea 16: `b.HasKey(x => x.Id);` → define explícitamente la clave primaria de la entidad.

Línea 17: `b.HasAlternateKey(x => x.Codigo).HasName("AK_Aleaciones_Codigo");` → define una clave alternativa para el identificador natural y permite referenciarlo como principal.

Línea 18: `b.Property(x => x.Nombre).IsRequired().HasMaxLength(100);` → marca la propiedad como requerida y limita su longitud en el esquema.

Línea 19: `b.Property(x => x.Codigo).IsRequired().HasMaxLength(20);` → marca la propiedad como requerida y limita su longitud en el esquema.

Línea 20: `b.Property(x => x.Descripcion).HasMaxLength(500);` → limita la longitud máxima de la columna correspondiente.

Línea 21: `b.HasIndex(x => x.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");` → crea y nombra un índice sobre la propiedad indicada.

Línea 22: `b.HasIndex(x => new { x.PorcentajeCarbono, x.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");` → crea un índice compuesto sobre las propiedades indicadas.

Línea 23: `b.HasQueryFilter(x => !x.IsDeleted);` → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.

Línea 24: } → cierra el bloque de código actual.

Línea 25: } → cierra el bloque de código actual.

Línea 27: `public sealed class EstadoOrdenConfiguration : IEntityTypeConfiguration<EstadoOrden>`
→ declara la configuración Fluent API de EstadoOrden.

Línea 28: { → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 29: `public void Configure(EntityTypeBuilder<EstadoOrden> b)` → implementa el método Configure de IEntityTypeConfiguration para describir el mapeo de la entidad.

Línea 30: { → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 31: `b.ToTable("EstadosOrden");` → asigna la entidad a la tabla SQL indicada; cuando incluye lambda, configura además restricciones de tabla.

Línea 32: `b.HasKey(x => x.Id);` → define explícitamente la clave primaria de la entidad.

Línea 33: `b.Property(x => x.Nombre).IsRequired().HasMaxLength(50);` → marca la propiedad como requerida y limita su longitud en el esquema.

Línea 34: `b.Property(x => x.Descripcion).HasMaxLength(250);` → limita la longitud máxima de la columna correspondiente.

Línea 35: `b.Property(x => x.Activo).HasDefaultValue(true);` → configura un valor por defecto de base de datos para la propiedad.

Línea 36: `b.HasQueryFilter(x => !x.IsDeleted && x.Activo);` → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.

Línea 37: } → cierra el bloque de código actual.

Línea 38: } → cierra el bloque de código actual.

Línea 40: `public sealed class DetalleOrdenConfiguration : IEntityTypeConfiguration<DetalleOrden>`
→ declara la configuración Fluent API de DetalleOrden.

Línea 41: { → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 42: `public void Configure(EntityTypeBuilder<DetalleOrden> b)` → implementa el método Configure de IEntityTypeConfiguration para describir el mapeo de la entidad.

Línea 43: { → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 44: `b.ToTable("DetallesOrden");` → asigna la entidad a la tabla SQL indicada; cuando incluye lambda, configura además restricciones de tabla.

Línea 45: `b.HasKey(x => x.Id);` → define explícitamente la clave primaria de la entidad.

Línea 46: `b.Property(x => x.ComposicionQuimica).IsRequired().HasMaxLength(200);` → marca la propiedad como requerida y limita su longitud en el esquema.

Línea 47: `b.Property(x => x.Notas).HasMaxLength(500);` → limita la longitud máxima de la columna correspondiente.

Línea 48: `b.HasOne(x => x.Orden).WithOne(x => x.Detalle).HasForeignKey<DetalleOrden>(x => x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();` → configura una relación uno-a-uno y encadena su clave foránea y comportamiento de borrado.

Línea 49: `b.HasQueryFilter(x => !x.Orden.IsDeleted && x.Orden.Estado != "Cancelada");` → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.

Línea 50: } → cierra el bloque de código actual.

Línea 51: } → cierra el bloque de código actual.

Línea 53: `public sealed class CertificadoCalidadConfiguration :`

`IEntityTypeConfiguration<CertificadoCalidad>` → declara la configuración Fluent API de `CertificadoCalidad`.

Línea 54: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 55: `public void Configure(EntityTypeBuilder<CertificadoCalidad> b)` → implementa el método `Configure` de `IEntityTypeConfiguration` para describir el mapeo de la entidad.

Línea 56: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 57: `b.ToTable("CertificadosCalidad");` → asigna la entidad a la tabla SQL indicada; cuando incluye `lambda`, configura además restricciones de tabla.

Línea 58: `b.HasKey(x => x.Id);` → define explícitamente la clave primaria de la entidad.

Línea 59: `b.HasAlternateKey(x =>`

`x.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");` → define una clave alternativa para el identificador natural y permite referenciarlo como principal.

Línea 60: `b.Property(x => x.NumeroCertificado).IsRequired().HasMaxLength(50);` → marca la propiedad como requerida y limita su longitud en el esquema.

Línea 61: `b.Property(x => x.OrganismoCertificador).IsRequired().HasMaxLength(100);` → marca la propiedad como requerida y limita su longitud en el esquema.

Línea 62: `b.HasIndex(x =>`

`x.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");` → crea y nombra un índice sobre la propiedad indicada.

Línea 63: `b.HasOne(x => x.Orden).WithOne(x => x.Certificado).HasForeignKey<CertificadoCalidad>(x => x.OrdenId).OnDelete(DeleteBehavior.Cascade).IsRequired();` → configura una relación uno-a-uno y encadena su clave foránea y comportamiento de borrado.

Línea 64: `b.HasQueryFilter(x => !x.Orden.IsDeleted && x.Orden.Estado != "Cancelada");` → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.

Línea 65: `}` → cierra el bloque de código actual.

Línea 66: `}` → cierra el bloque de código actual.

Línea 68: `public sealed class OrdenAleacionConfiguration :`

`IEntityTypeConfiguration<OrdenAleacion>` → declara la configuración Fluent API de `OrdenAleacion`.

Línea 69: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 70: `public void Configure(EntityTypeBuilder<OrdenAleacion> b)` → implementa el método `Configure` de `IEntityTypeConfiguration` para describir el mapeo de la entidad.

Línea 71: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 72: `b.ToTable("OrdenesAleaciones");` → asigna la entidad a la tabla SQL indicada; cuando incluye `lambda`, configura además restricciones de tabla.

Línea 73: `b.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });` → define la clave primaria compuesta de la entidad de unión con ambas claves foráneas.

Línea 74: `b.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");` → configura una propiedad cuyo valor por defecto se obtiene mediante una expresión SQL del servidor.

Línea 75: `b.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);` → configura precisión y escala para el valor decimal.

Línea 76: `b.Property(x =>`

`x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");` → configura un valor por defecto de base de datos para la propiedad.

Línea 77: `b.HasOne(x => x.Orden).WithMany(x => x.OrdenesAleaciones).HasForeignKey(x => x.OrdenFabricacionId).OnDelete(DeleteBehavior.Cascade);` → configura una relación uno-a-muchos y encadena la clave foránea y el comportamiento de borrado.

Línea 78: `b.HasOne(x => x.Aleacion).WithMany(x => x.OrdenesAleaciones).HasForeignKey(x => x.AleacionId).OnDelete(DeleteBehavior.Restrict);` → configura una relación uno-a-muchos y encadena la clave foránea y el comportamiento de borrado.

Línea 79: `b.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");` → crea y nombra un índice sobre la propiedad indicada.

Línea 80: `b.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa']").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");` → crea un índice filtrado para el subconjunto de filas indicado.

Línea 81: `b.HasQueryFilter(x => x.EstadoRelacion == "Activa" && !x.Orden.IsDeleted && !x.Aleacion.IsDeleted);` → aplica el filtro global que oculta registros eliminados o inactivos en las consultas normales.

Línea 82: `}` → cierra el bloque de código actual.

Línea 83: `}` → cierra el bloque de código actual.

src/AceriaData.Infrastructure/Repositories/Repositories.cs

Línea 1: `using AceriaData.Application.Interfaces;` → importa los puertos definidos por Application para que Infrastructure pueda implementarlos.

Línea 2: `using AceriaData.Domain.Entities;` → importa las entidades del dominio sin introducir dependencias de EF Core en Domain.

Línea 3: `using AceriaData.Infrastructure.Persistence;` → importa el DbContext y los componentes de persistencia de Infrastructure.

Línea 4: `using Microsoft.EntityFrameworkCore;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 6: `namespace AceriaData.Infrastructure.Repositories;` → declara el espacio de nombres AceriaData.Infrastructure.Repositories y sitúa el archivo en su capa arquitectónica.

Línea 8: `public sealed class OrdenRepository : IOrdenRepository` → declara el adaptador de Infrastructure que implementa IOrdenRepository.

Línea 9: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 10: `private readonly AceriaDbContext _context;` → mantiene la referencia al DbContext compartido por el adaptador dentro del ámbito DI.

Línea 11: `public OrdenRepository(AceriaDbContext context) => _context = context;` → inyecta AceriaDbContext en el adaptador de repositorio.

Línea 12: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave mediante DbSet.Find.

Línea 13: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(x => x.NumeroOrden == numeroOrden);` → implementa la búsqueda por número de orden mediante una consulta LINQ traducida a SQL.

Línea 14: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(x => x.Id).ToList();` → materializa las órdenes ordenadas por Id.

Línea 15: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra la orden como Added en el ChangeTracker sin ejecutar todavía SaveChanges.

Línea 16: `public void Eliminar(OrdenFabricacion orden) =>`

`_context.OrdenesFabricacion.Remove(orden);` → marca la orden como Deleted en el ChangeTracker; la escritura se confirma al guardar.

Línea 17: `}` → cierra el bloque de código actual.

Línea 19: `public sealed class UnidadDeTrabajo : IUnidadDeTrabajo` → declara el adaptador que implementa IUnidadDeTrabajo y concentra SaveChanges.

Línea 20: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 21: `private readonly AceriaDbContext _context;` → mantiene la referencia al DbContext compartido por el adaptador dentro del ámbito DI.

Línea 22: `private IOrdenRepositorio? _ordenes;` → mantiene la instancia perezosa del repositorio de órdenes.

Línea 23: `public UnidadDeTrabajo(AceriaDbContext context) => _context = context;` → inyecta el mismo AceriaDbContext en la unidad de trabajo.

Línea 24: `public IOrdenRepositorio Ordenes => _ordenes ??= new OrdenRepositorio(_context);` → crea perezosamente OrdenRepositorio y reutiliza el DbContext de la unidad de trabajo.

Línea 25: `public int Guardar() => _context.SaveChanges();` → confirma en SQL Server todos los cambios pendientes del DbContext.

Línea 26: `public void Dispose() => _context.Dispose();` → libera el DbContext cuando termina la unidad de trabajo.

Línea 27: `}` → cierra el bloque de código actual.

src/AceriaData.Infrastructure/DependencyInjection.cs

Línea 1: `using AceriaData.Application.Interfaces;` → importa los puertos definidos por Application para que Infrastructure pueda implementarlos.

Línea 2: `using AceriaData.Infrastructure.Persistence;` → importa el DbContext y los componentes de persistencia de Infrastructure.

Línea 3: `using AceriaData.Infrastructure.Repositories;` → importa los adaptadores concretos que implementan los puertos de Application.

Línea 4: `using Microsoft.EntityFrameworkCore;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el contenedor de dependencias y sus métodos de registro/resolución.

Línea 7: `namespace AceriaData.Infrastructure;` → declara el espacio de nombres AceriaData.Infrastructure y sitúa el archivo en su capa arquitectónica.

Línea 9: `public static class DependencyInjection` → declara una clase estática usada como contenedor de extensiones de registro.

Línea 10: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 11: `public static IServiceCollection AddAceriaInfrastructure(this IServiceCollection services, string connectionString)` → define el método de extensión que encapsula todos los registros de Infrastructure.

Línea 12: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 13: `services.AddDbContext<AceriaDbContext>(o => o.UseSqlServer(connectionString));` → registra AceriaDbContext con SQL Server usando la cadena recibida por el composition root.

Línea 14: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → vincula IOrdenRepositorio con OrdenRepositorio mediante ciclo de vida scoped.

Línea 15: `services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();` → vincula IUnidadDeTrabajo con UnidadDeTrabajo mediante ciclo de vida scoped.

Línea 16: `return services;` → devuelve la colección para permitir encadenar el registro de servicios.

Línea 17: `}` → cierra el bloque de código actual.

Línea 18: `}` → cierra el bloque de código actual.

src/AceriaData.Console/Program.cs

Línea 1: `using AceriaData.Application.UseCases;` → importa el caso de uso que la capa de entrada va a resolver y ejecutar.

Línea 2: `using AceriaData.Infrastructure;` → importa el espacio de nombres requerido por los tipos usados en este archivo.

Línea 3: `using AceriaData.Infrastructure.Persistence;` → importa el DbContext y los componentes de persistencia de Infrastructure.

Línea 4: `using Microsoft.EntityFrameworkCore;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 5: `using Microsoft.Extensions.Configuration;` → importa la API de configuración usada para leer appsettings.json y variables de entorno.

Línea 6: `using Microsoft.Extensions.DependencyInjection;` → importa el contenedor de dependencias y sus métodos de registro/resolución.

Línea 8: `var configuration = new ConfigurationBuilder()` → inicia la configuración de la aplicación de consola.

Línea 9: `.SetBasePath(AppContext.BaseDirectory)` → establece la carpeta base desde la que se localizará appsettings.json.

Línea 10: `.AddJsonFile("appsettings.json", optional: false)` → carga appsettings.json como fuente obligatoria de configuración.

Línea 11: `.AddEnvironmentVariables()` → añade variables de entorno, que pueden complementar o sobrescribir la configuración.

Línea 12: `.Build();` → construye el objeto IConfiguration definitivo.

Línea 14: `var cs = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena AceriaDB desde la configuración.

Línea 15: `?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");` → falla de forma explícita si la cadena de conexión no está configurada.

Línea 17: `var services = new ServiceCollection();` → crea la colección de servicios del composition root.

Línea 18: `services.AddAceriaInfrastructure(cs);` → registra en Console todos los adaptadores de Infrastructure mediante su extensión.

Línea 19: `services.AddScoped<CrearOrdenUseCase>();` → registra el caso de uso para resolverlo dentro del mismo ámbito.

Línea 21: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true, ValidateScopes = true });` → construye el proveedor de inyección de dependencias con validación y garantiza su liberación automática al terminar.

Línea 22: `using var scope = provider.CreateScope();` → crea un ámbito scoped para resolver DbContext, repositorios y casos de uso, y lo libera automáticamente al terminar.

Línea 23: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve el DbContext registrado en Infrastructure.

Línea 24: `context.Database.EnsureDeleted();` → elimina la base del laboratorio para que la ejecución del laboratorio sea reproducible; no sustituye a Migrations.

Línea 25: `context.Database.Migrate();` → aplica el historial real de migraciones hasta el esquema 2.12.

Línea 27: `var crear = scope.ServiceProvider.GetRequiredService<CrearOrdenUseCase>();` → resuelve el caso de uso desde el contenedor.

Línea 28: `crear.Ejecutar("0F-M2-HEX-0001", "Cliente Arquitectura");` → ejecuta el caso de uso y persiste la orden a través de los puertos de Application.

Línea 29: `var orden = context.OrdenesFabricacion.Single();` → consulta la única orden creada para verificar el resultado del escenario práctico.

Línea 30: `Console.WriteLine($"2.12 OK | {orden.NumeroOrden} | {orden.Cliente}");` → imprime el marcador 2.12 OK y los datos persistidos que confirma el resultado esperado del punto.

src/AceriaData.Infrastructure/Persistence/AceriaDesignTimeDbContextFactory.cs

Línea 1: `using Microsoft.EntityFrameworkCore;` → importa las APIs de Entity Framework Core necesarias en Infrastructure o en el composition root.

Línea 2: `using Microsoft.EntityFrameworkCore.Design;` → importa el contrato de factory de diseño utilizado por las herramientas dotnet ef.

Línea 4: `namespace AceriaData.Infrastructure.Persistence;` → declara el espacio de nombres AceriaData.Infrastructure.Persistence y sitúa el archivo en su capa arquitectónica.

Línea 6: `public sealed class AceriaDesignTimeDbContextFactory : IDesignTimeDbContextFactory<AceriaDbContext>` → declara la factory de diseño que permite a dotnet ef construir AceriaDbContext sin arrancar Console.

Línea 7: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 8: `public AceriaDbContext CreateDbContext(string[] args)` → implementa el contrato de diseño que usan las herramientas EF Core.

Línea 9: `{` → abre el bloque correspondiente a la declaración o instrucción anterior.

Línea 10: `var options = new DbContextOptionsBuilder<AceriaDbContext>()` → inicia la construcción de DbContextOptions para el contexto de diseño.

Línea 11: `.UseSqlServer("Server=(localdb)\MSSQLLocalDB;Database=AceriaDB;Trusted_Connection=True;TrustServerCertificate=True;")` → configura SQL Server LocalDB y la base AceriaDB para las operaciones de diseño.

Línea 12: `.Options;` → extrae las opciones ya configuradas del builder.

Línea 13: `return new AceriaDbContext(options);` → crea y devuelve AceriaDbContext con esas opciones para dotnet ef.

Línea 14: `}` → cierra el bloque de código actual.

Línea 15: `}` → cierra el bloque de código actual.

Errores comunes

Error	Causa	Solución
Domain referencia EF Core	Se movieron entidades junto con tipos de persistencia	Mantener Domain como clases de negocio sin paquete EF Core.
Application usa DbContext	Se filtró una dependencia de infraestructura	Depender de IOrdenRepositorio/ IUnidadDeTrabajo.

Error	Causa	Solución
EF Tools no encuentra el contexto	Startup o project incorrectos	Indicar <code>--project Infrastructure --startup-project Console</code> .
La consola no resuelve servicios	Falta AddAceriaInfrastructure	Registrar la infraestructura antes de BuildServiceProvider.

Retorno resuelto

Retorno: Crear una orden desde CrearOrdenUseCase y comprobar que Application no referencia Microsoft.EntityFrameworkCore.

Solución: revisar los `.csproj`: Domain y Application no contienen referencias a `Microsoft.EntityFrameworkCore`; Infrastructure sí contiene `Microsoft.EntityFrameworkCore.SqlServer` y `Design`. Ejecutar después la solución y comprobar el marcador `2.12 OK`.

Analogía final

Clean Architecture separa el conocimiento del negocio de la tecnología de persistencia: EF Core pasa a ser un adaptador reemplazable alrededor del núcleo.

Resultado esperado

M2 termina con una solución multiproyecto compilable y ejecutable, el mismo modelo acumulado, EF Core aislado en Infrastructure y una aplicación que accede a persistencia mediante puertos definidos fuera de la infraestructura.

Conexión con el siguiente módulo

M03/PROYECTO03.1 deberá partir de este estado completo de 2.12.